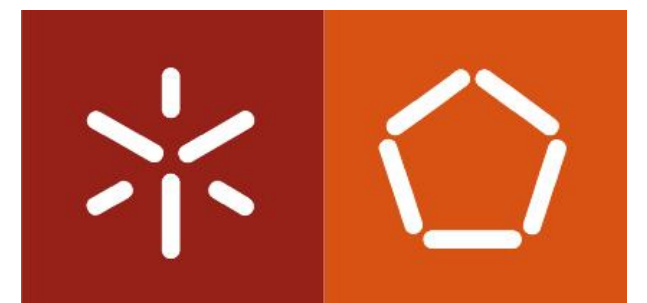


Operating Systems

(Sistemas Operativos)

Guide 6: Dup



File Interface

Duplicating file descriptors

#include <unistd.h>

- *int* ***dup***(*int fd*)
 - **fd**: the file descriptor
 - Returns: a new file descriptor referring to the **same open file table entry as *fd***
(-1 on errors)

Obs:

- chooses lowest-numbered available descriptor
- preserves original mode and position of *fd*

For more information: *\$ man dup*

#include <unistd.h>

- *int* ***dup2***(*int oldfd*, *int newfd*)
 - **oldfd**: the file descriptor
 - **newfd**: the file descriptor to refer to the same open file table entry as *oldfd*
 - Returns: the new file descriptor (*newfd*) or -1 on errors

Obs:

- **if *newfd* is open, *dup2* closes it implicitly**
- preserves original mode and position of *oldfd*

For more information: *\$ man dup*

File Interface

Duplicating file descriptors

`#include <unistd.h>`

- `int dup(int fd)`

- **fd**: the file descriptor

- Returns: a new file descriptor referring to the same open file table entry as *fd* (-1 on errors)

Obs:

- chooses lowest-numbered available descriptor
- preserves original mode and position of *fd*

For more information: `$ man dup`

`#include <unistd.h>`

- `int dup2(int oldfd, int newfd)`

- **oldfd**: the file descriptor

- **newfd**: the file descriptor to refer to the same open file table entry as *oldfd*

- Returns: the new file descriptor (*newfd*) or -1 on errors

Obs:

- if *newfd* is open, **dup2** closes it implicitly
- preserves original mode and position of *oldfd*

For more information: `$ man dup`

**Imagine we want to redirect
all writes done to STDOUT to a file.**

How could we do this?

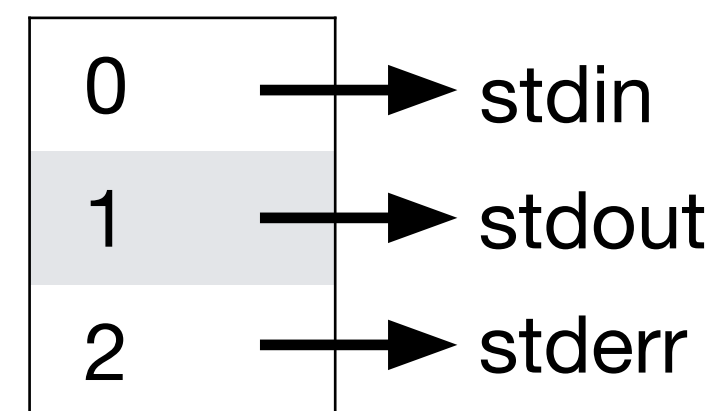
File Interface

Example 1: redirecting STDOUT to a file



```
1 int main() {  
    int fd = open("foo.txt", O_CREAT |  
2 O_APPEND | O_WRONLY, 0600);  
  
3    printf("Opened fd=%d\n", fd);  
4    dup2(fd, 1);  
5    close(fd);  
6    printf("Redirected stdout to  
    foo.txt\n");  
7    // ...  
8    return 0;  
9 }
```

FD array of process A



File Interface

Example 1: redirecting STDOUT to a file

```
1 int main() {  
2     int fd = open("foo.txt", O_CREAT |  
3     O_APPEND | O_WRONLY, 0600);  
4     printf("Opened fd=%d\n", fd);  
5     dup2(fd, 1);  
6     close(fd);  
7     printf("Redirected stdout to  
8     foo.txt\n");  
9     // ...  
10    return 0;  
11 }
```

Opening file *foo.txt* for writing ...

FD array of process A

0	→	stdin
1	→	stdout
2	→	stderr
3	→	

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	0
#REF	1
...	

File Interface

Example 1: redirecting STDOUT to a file

```
1 int main() {  
    int fd = open("foo.txt", O_CREAT |  
2 O_APPEND | O_WRONLY, 0600);  
    printf("Opened fd=%d\n", fd);  
3    dup2(fd, 1);  
4    close(fd);  
5    printf("Redirected stdout to  
6    foo.txt\n");  
7    // ...  
8    return 0;  
9 }
```

Writing to STDOUT ...

FD array of process A

0	→	stdin
1	→	stdout
2	→	stderr
3	→	

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	0
#REF	1
...	

At this point, a ***write(1,...)*** or a ***printf*** will result in data being written to the **stdout (display)**!

File Interface

Example 1: redirecting STDOUT to a file

```
1 int main() {
2     int fd = open("foo.txt", O_CREAT |
3     O_APPEND | O_WRONLY, 0600);
4     printf("Opened fd=%d\n", fd);
5     dup2(fd, 1);
6     close(fd);
7     printf("Redirected stdout to
8     foo.txt\n");
9     // ...
10    return 0;
11 }
```

Redirecting STDOUT to file *foo.txt* ...

FD array of process A

0	→ stdin
1	→
2	→ stderr
3	→

Entry at the open file table

MODE	W
OFFSET	0
#REF	2
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	0
#REF	1
...	

As of now, a ***write***(1,...) or a ***printf*** will result in data being written to **file foo.txt!**

File Interface

Example 1: redirecting STDOUT to a file

```
1 int main() {  
2     int fd = open("foo.txt", O_CREAT |  
3     O_APPEND | O_WRONLY, 0600);  
4     printf("Opened fd=%d\n", fd);  
5     dup2(fd, 1);  
6     close(fd);  
7     printf("Redirected stdout to  
8     foo.txt\n");  
9     // ...  
10    return 0;  
11 }
```

Closing unused file descriptors ...

FD array of process A

0	→ stdin
1	
2	→ stderr

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	0
#REF	1
...	

File Interface

Example 1: redirecting STDOUT to a file

```
1 int main() {  
2     int fd = open("foo.txt", O_CREAT |  
3     O_APPEND | O_WRONLY, 0600);  
4     printf("Opened fd=%d\n", fd);  
5     dup2(fd, 1);  
6     close(fd);  
7     printf("Redirected stdout to  
8     foo.txt\n");  
9     // ...  
10    return 0;  
11 }
```

Writing to STDOUT ...

FD array of process A

0	→ stdin
1	→
2	→ stderr

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	30
#REF	1
...	

The ***printf*** will result in a write of 30 bytes to **file *foo.txt***.

File Interface

Example 1: redirecting STDOUT to a file

```
1 int main() {  
    int fd = open("foo.txt", O_CREAT |  
2 O_APPEND | O_WRONLY, 0600);  
3    printf("Opened fd=%d\n", fd);  
4    dup2(fd, 1);  
5    close(fd);  
6    printf("Redirected stdout to  
    foo.txt\n");  
7    // ...  
8    return 0;  
9 }
```

Closing unused file descriptors ...

**What if we wanted to revert
STDOUT to its previous value?**

FD array of process A

Entry at the open file table

inode for file foo.txt

1
2

stderr

OFFSET	0
#REF	1
INODE	

I-NUMBER	100
SIZE	30
#REF	1
...	

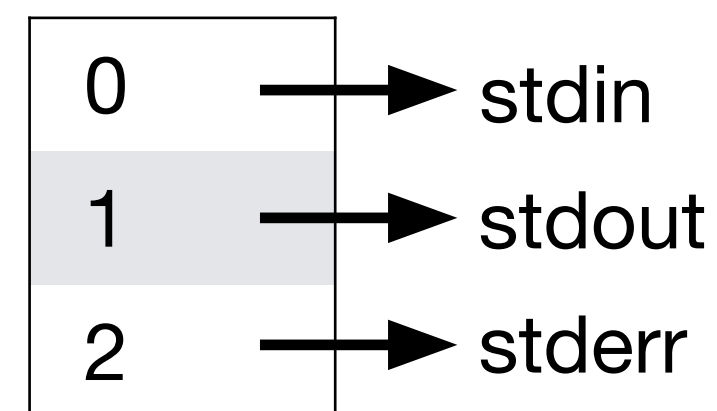
File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value



```
1 int main() {
2     int fd = open("foo.txt", O_CREAT |
3     O_APPEND | O_WRONLY, 0600);
4     int original_stdout = dup(1);
5     dup2(fd, 1);
6     close(fd);
7     printf("Redirected stdout to
8     foo.txt\n");
9     // ...
10    dup2(original_stdout, 1);
11    close(original_stdout);
12    printf("Restored STDOUT\n");
13    return 0;
14 }
```

FD array of process A



File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1 int main() {  
2     int fd = open("foo.txt", O_CREAT |  
3     O_APPEND | O_WRONLY, 0600);  
4     int original_stdout = dup(1);  
5     dup2(fd, 1);  
6     close(fd);  
7     printf("Redirected stdout to  
8     foo.txt\n");  
9     // ...  
10    dup2(original_stdout, 1);  
11    close(original_stdout);  
12    printf("Restored STDOUT\n");  
13    return 0;  
14 }
```

Opening file *foo.txt* for writing ...

FD array of process A

0	→	stdin
1	→	stdout
2	→	stderr
3	→	

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	0
#REF	1
...	

File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1 int main() {
2     int fd = open("foo.txt", O_CREAT |
3     O_APPEND | O_WRONLY, 0600);
4     int original_stdout = dup(1);
5     dup2(fd, 1);
6     close(fd);
7     printf("Redirected stdout to
8     foo.txt\n");
9     // ...
10    dup2(original_stdout, 1);
11    close(original_stdout);
12    printf("Restored STDOUT\n");
13    return 0;
14 }
```

Creating a copy of STDOUT descriptor ...

FD array of process A

0	→	stdin
1	→	stdout
2	→	stderr
3	→	
4	→	

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	0
#REF	1
...	

File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1 int main() {
2     int fd = open("foo.txt", O_CREAT |
3     O_APPEND | O_WRONLY, 0600);
4     int original_stdout = dup(1);
5     dup2(fd, 1);
6     close(fd);
7     printf("Redirected stdout to
8     foo.txt\n");
9     // ...
10    dup2(original_stdout, 1);
11    close(original_stdout);
12    printf("Restored STDOUT\n");
13    return 0;
14 }
```

Redirecting STDOUT to file *foo.txt* ...

FD array of process A

0	→ stdin
1	→
2	→ stderr
3	→
4	→ stdout

Entry at the open file table

MODE	W
OFFSET	0
#REF	2
INODE	

inode for file *foo.txt*

I-NUMBER	100
SIZE	0
#REF	1
...	

As of now, a ***write*(1,...)** or a ***printf*** will result in data being written to **file *foo.txt*!**

File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1 int main() {
2     int fd = open("foo.txt", O_CREAT |
3     O_APPEND | O_WRONLY, 0600);
4     int original_stdout = dup(1);
5     dup2(fd, 1);
6     close(fd);
7     printf("Redirected stdout to
8     foo.txt\n");
9     // ...
10    dup2(original_stdout, 1);
11    close(original_stdout);
12    printf("Restored STDOUT\n");
13    return 0;
14 }
```

Closing unused file descriptors ...

FD array of process A

0	→ stdin
1	
2	→ stderr
4	→ stdout

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	0
#REF	1
...	

File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1 int main() {
2     int fd = open("foo.txt", O_CREAT |
3     O_APPEND | O_WRONLY, 0600);
4     int original_stdout = dup(1);
5     dup2(fd, 1);
6     close(fd);
7     printf("Redirected stdout to
8     foo.txt\n");
9     // ...
10    dup2(original_stdout, 1);
11    close(original_stdout);
12    printf("Restored STDOUT\n");
13    return 0;
14 }
```

Writing to STDOUT ...

FD array of process A

0	→	stdin
1	→	
2	→	stderr
4	→	stdout

Entry at the open file table

MODE	W
OFFSET	0
#REF	1
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	30
#REF	1
...	

The ***printf*** will result in a write of 30 bytes to **file *foo.txt***.

File Interface

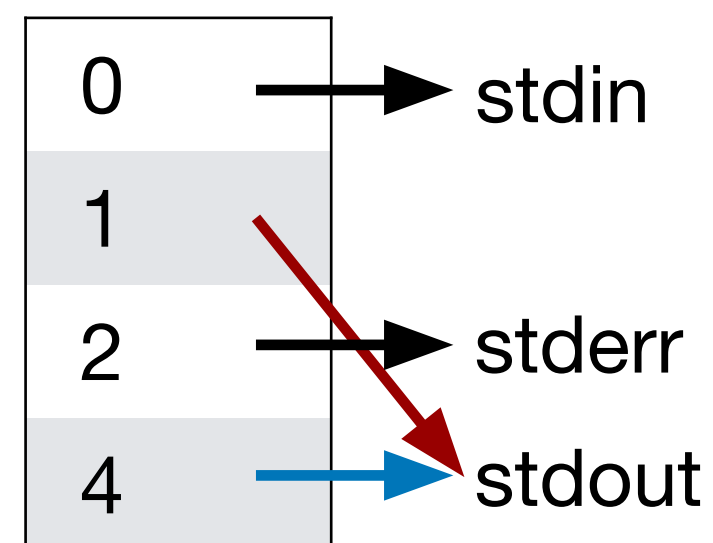
Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1  int main() {
2      int fd = open("foo.txt", O_CREAT |
3      O_APPEND | O_WRONLY, 0600);
4      int original_stdout = dup(1);
5      dup2(fd, 1);
6      close(fd);
7      printf("Redirected stdout to
8      foo.txt\n");
9      // ...
10     dup2(original_stdout, 1);
11     close(original_stdout);
12     printf("Restored STDOUT\n");
13     return 0;
14 }
```



Redirecting STDOUT to its original value ...

FD array of process A



inode for file foo.txt

I-NUMBER	100
SIZE	30
#REF	0
...	

As of now, a ***write***(1,...) or a ***printf*** will result in data being written again to **stdout (display)**!

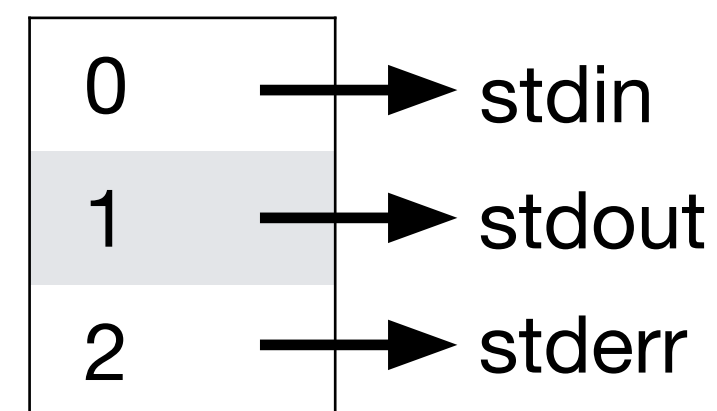
File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1  int main() {
2      int fd = open("foo.txt", O_CREAT |
3      O_APPEND | O_WRONLY, 0600);
4      int original_stdout = dup(1);
5      dup2(fd, 1);
6      close(fd);
7      printf("Redirected stdout to
8      foo.txt\n");
9      // ...
10     dup2(original_stdout, 1);
11     close(original_stdout);
12     printf("Restored STDOUT\n");
13     return 0;
14 }
```

Closing unused file descriptors ...

FD array of process A



inode for file foo.txt

I-NUMBER	100
SIZE	30
#REF	0
...	

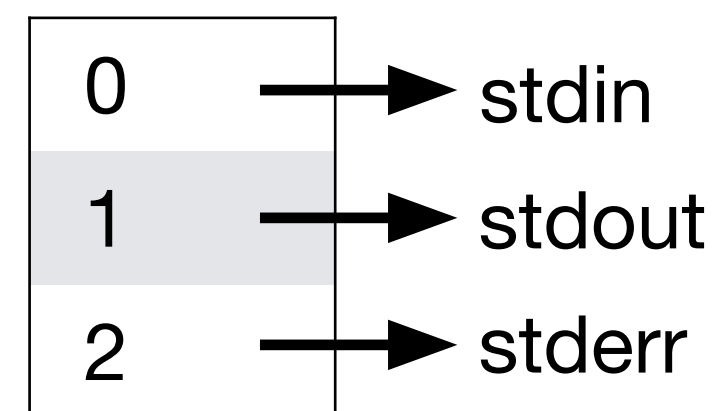
File Interface

Example 2: redirecting STDOUT to a file, then restoring to its previous value

```
1  int main() {
2      int fd = open("foo.txt", O_CREAT |
3      O_APPEND | O_WRONLY, 0600);
4      int original_stdout = dup(1);
5      dup2(fd, 1);
6      close(fd);
7      printf("Redirected stdout to
8      foo.txt\n");
9      // ...
10     dup2(original_stdout, 1);
11     close(original_stdout);
12     printf("Restored STDOUT\n");
13     return 0;
14 }
```

Writing to STDOUT ...

FD array of process A



inode for file foo.txt

I-NUMBER	100
SIZE	30
#REF	0
...	

The ***printf*** will result in a write of 17 bytes to **stdout (display)**!.

File Interface

Important remarks – redirecting stdout to a file

- The memory buffer size changes from a single line to the file system's default size (typically 4KB).

- `printf()`, etc. no longer perform line buffering.

- Use `fflush(stdout)` to flush buffered data to the file (per operation)

or

- Use `setbuf(stdout, NULL)` to disable buffering (globally)

`dup2(fd, 1)`

FD array of process A

0	→ stdin
1	
2	→ stderr
3	

Entry at the open file table

MODE	W
OFFSET	0
#REF	2
INODE	

inode for file foo.txt

I-NUMBER	100
SIZE	30
#REF	1
...	